

Rest API

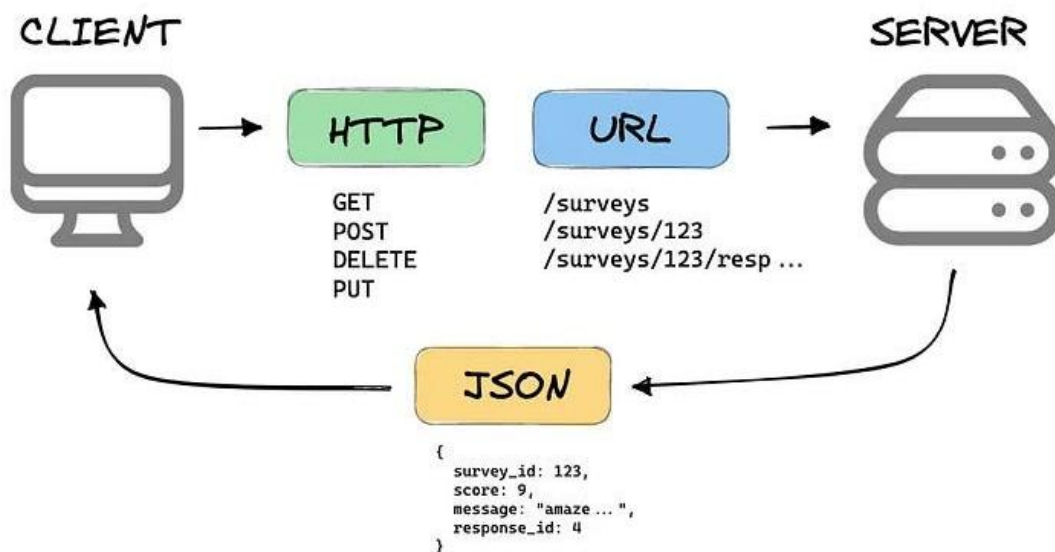
Introduction to RESTful Web Services :-

- RESTful Web Services provide a simple, scalable, and stateless way to build web APIs.
- REST, introduced by Roy Fielding, is designed to efficiently use HTTP.
- It is lighter than SOAP and does not require strict messaging protocols.
- REST supports data formats like JSON and XML, with JSON being the most widely used.

What is a Rest API ?

- REST API stands for Representational State Transfer API.
- It enables communication between different systems over the internet.
- REST APIs send requests and receive responses between client and server.
- They typically use JSON format for data exchange.

WHAT IS A REST API?



- A client sends a request to the server through a web URL using an HTTP method.
- The server responds with a resource like HTML, XML, Image, or JSON.
- JSON is the most commonly used format in modern web services.
- HTTP methods map to CRUD operations: Create, Read, Update, Delete.

Key Concepts :-

1. **Resource:** Any object, entity, or service that can be accessed via a URI.
2. **Stateless Communication:** Each HTTP request contains all the information needed to process it.
3. **Representations:** Resources can be represented in different formats (JSON, XML, HTML, PDF, etc.).
4. **HTTP Verbs:** REST leverages standard HTTP methods for CRUD operations.

HTTP Methods :-

The main methods of HTTP we build web services for are:

1. **GET:** Reads existing data.
2. **PUT:** Updates existing data.
3. **POST:** Creates new data.
4. **DELETE:** Deletes the data.

1. GET – Read Resource

- Retrieves data without a request body.
- Can fetch a specific resource using an ID or a collection without parameters.

```
@GetMapping("/user/{userId}")
public ResponseEntity<UserEntity> getUser(@PathVariable int userId) {
    UserEntity user = userService.getUser(userId);
    return ResponseEntity.ok(user);
}
```

2. POST – Create Resource

- Creates a new resource using a request body

```
@PostMapping("/user")
public ResponseEntity<String> addUser(@RequestBody UserEntity user) {
    userService.saveOrUpdate(user);
    return ResponseEntity.status(HttpStatus.CREATED).body("User created successfully");
}
```

3. PUT – Update Resource

- Updates an existing resource identified by ID.

```
@PutMapping("/user/{userId}")
public ResponseEntity<String> updateUser(@PathVariable int userId, @RequestBody UserEntity user) {
    userService.saveOrUpdate(user);
    return ResponseEntity.ok("User updated successfully");
}
```

4. DELETE – Remove Resource

- Deletes a single or multiple resources based on parameters.

```
@DeleteMapping("/user/{userId}")
public ResponseEntity<String> deleteUser(@PathVariable int userId) {
    userService.deleteUser(userId);
    return ResponseEntity.ok("User deleted successfully");
}
```

HTTP Status Codes :-

1. 200: Success
2. 201: Created
3. 401: Unauthorized
4. 404: Resource Not Found
5. 500: Server Error

REST APIs rely on these codes to communicate the result of client requests.

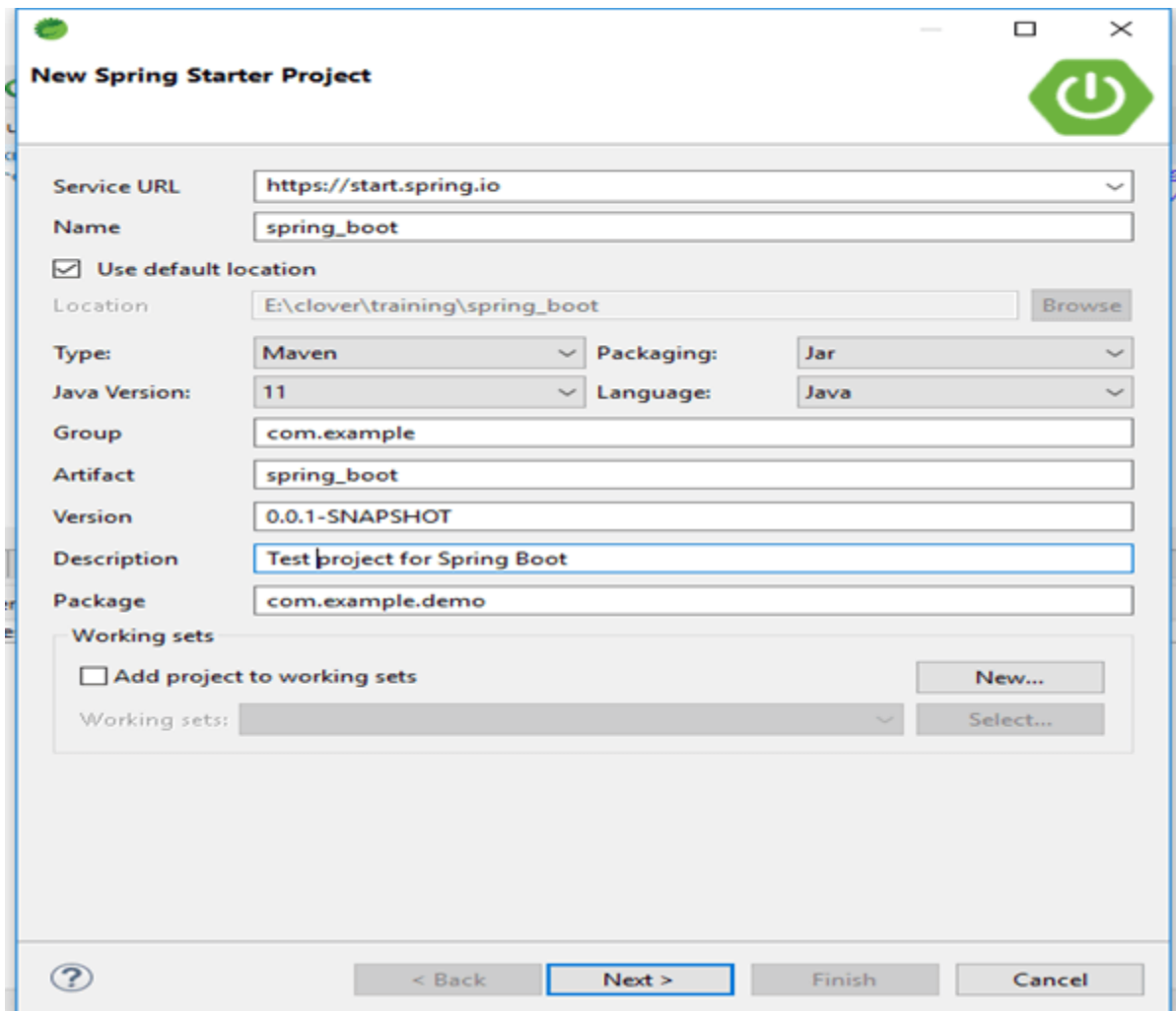
Real-World Examples of REST APIs :-

1. **Social Media:** Used for login, posting, and sharing content across platforms like Facebook and Twitter.
2. **E-Commerce:** Handles products, orders, payments, and customer data.
3. **Geolocation:** Provides GPS tracking, real-time location, and nearby place searches.
4. **Weather Apps:** Fetch real-time weather data and forecasts from external services.

Step-by-Step Implementation of REST API in Spring Boot :-

Step 1: Create the Spring Boot Project

1. Open **Eclipse IDE**.
2. Go to **File → New → Spring Starter Project**.
3. Enter the project details:
 - Project Name: **Spring_Boot_Rest_API**
 - Group: **com.example.demo**
 - Artifact: **demo**
 - Packaging: **Jar**
 - Java Version: **17+**
4. Click **Next** → Add dependencies → Select **Spring Web**.
5. Click **Finish** → Eclipse will create and load the Spring Boot project automatically.



The screenshot shows the 'New Spring Starter Project' dialog box in Eclipse IDE. The dialog is titled 'New Spring Starter Project' and features a green power button icon in the top right corner. The fields are filled with the following values:

- Service URL: `https://start.spring.io`
- Name: `spring_boot`
- ☒ Use default location
- Location: `E:\clover\training\spring_boot` (with a 'Browse' button)
- Type: `Maven` (dropdown)
- Packaging: `Jar` (dropdown)
- Java Version: `11` (dropdown)
- Language: `Java` (dropdown)
- Group: `com.example`
- Artifact: `spring_boot`
- Version: `0.0.1-SNAPSHOT`
- Description: `Test project for Spring Boot`
- Package: `com.example.demo`

At the bottom, there is a 'Working sets' section with an unchecked checkbox 'Add project to working sets', a 'New...' button, and a 'Select...' button. The bottom navigation bar includes a help icon, '< Back', 'Next >' (highlighted), 'Finish', and 'Cancel' buttons.



—

□

×

New Spring Starter Project Dependencies



Spring Boot Version: 2.5.4

Available:

Type to search dependencies

▶ Developer Tools

▶ Google Cloud Platform

▶ I/O

▶ Messaging

▶ Microsoft Azure

▶ NoSQL

▶ Observability

▶ Ops

▶ SQL

▶ Security

▶ Spring Cloud

▶ Spring Cloud Circuit Breaker

▶ Spring Cloud Config

Selected:

?

< Back

Next >

Finish

Cancel

Step 2: Define the Employee Entity

Create a simple Student class inside com.example.demo:

```
package com.example.demo;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column
    private String fullname;

    @Column(unique=true)
    private String email;

    @Column
    private String password;

    @Column
    private String address;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getFullname() {
        return fullname;
    }

    public void setFullname(String fullname) {
        this.fullname = fullname;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    @Override
    public String toString() {
        return "Student [id=" + id + ", fullname=" + fullname + ", email=" + email + ", password=" + password
            + ", address=" + address + "]";
    }
}
```

Step 3: Create a StudentService Class

Create a class to hold a list of employees.

```
package com.example.demo;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class StudentService {

    @Autowired
    StudentRepo sr;

    public void registration(Student s)
    {
        sr.save(s);    //insert
    }

    public List<Student> getallstudents(){
        return sr.findAll();    //select * from student
    }
}
```

Step 4: Create the StudentRepo Class

The StudentRepo class will handle adding and retrieving employees.

```
package com.example.demo;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface StudentRepo extends JpaRepository<Student, Long> {

}
```

Step 5: Create the Controller

The controller contains the REST API endpoints.

```
package com.example.demo;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
@CrossOrigin("*")
public class MyController {

    @GetMapping("/")
    public ResponseEntity<> test()
    {
        return ResponseEntity.status(HttpStatus.ACCEPTED).body("Tested ok");
    }

    @Autowired
    StudentService ss;

    @PostMapping("/register")
    public ResponseEntity<> registration(@RequestBody Student s)
    {
        try {
            ss.registration(s);
            return ResponseEntity.status(HttpStatus.ACCEPTED).body("Registration Successfully Completed");
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("Fail");
        }
    }

    @GetMapping("/viewstudents")
    public ResponseEntity<List<Student>> viewstudents()
    {
        try {
            List<Student> data=ss.getallstudents();
            return ResponseEntity.ok(data);
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).build();
        }
    }
}
```

Open `SpringBootApplication.java` (already generated by Spring Boot) and run it:

```

package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringBootRestApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringBootRestApiApplication.class, args);
    }

}

:: Spring Boot ::                (v4.0.0)

2025-12-09T14:57:14.134+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] c.e.demo.SpringBootRestApiApplication
\target\classes started by sagar in C:\Users\sagar\workspace\Spring_Boot_Rest_API)
2025-12-09T14:57:14.145+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] c.e.demo.SpringBootRestApiApplication
2025-12-09T14:57:14.277+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor
2025-12-09T14:57:14.277+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor
2025-12-09T14:57:16.079+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] .s.d.r.c.RepositoryConfigurationDelegate
2025-12-09T14:57:16.131+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] .s.d.r.c.RepositoryConfigurationDelegate
2025-12-09T14:57:17.347+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] o.s.boot.tomcat.TomcatWebServer
2025-12-09T14:57:17.384+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] o.apache.catalina.core.StandardService
2025-12-09T14:57:17.385+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] o.apache.catalina.core.StandardEngine
2025-12-09T14:57:17.544+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] b.w.c.s.WebApplicationContextInitializer
2025-12-09T14:57:17.910+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] o.hibernate.jpa.internal.util.LogHelper
2025-12-09T14:57:18.056+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] org.hibernate.Version
2025-12-09T14:57:19.628+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo
2025-12-09T14:57:19.713+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] com.zaxxer.hikari.HikariDataSource
2025-12-09T14:57:20.408+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] com.zaxxer.hikari.pool.HikariPool
2025-12-09T14:57:20.413+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] com.zaxxer.hikari.HikariDataSource
2025-12-09T14:57:20.503+05:30 WARN 159048 --- [Spring_Boot_Rest_API] [ restartedMain] org.hibernate.orm.deprecation
it will be selected by default)
2025-12-09T14:57:20.575+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] org.hibernate.orm.connections.pooling
Database JDBC URL [jdbc:postgresql://localhost:5433/spring_boot_API]
Database driver: PostgreSQL JDBC Driver
Database dialect: PostgreSQLDialect
Database version: 18.0
Default catalog/schema: spring_boot_API/public
Autocommit mode: undefined/unknown
Isolation level: READ_COMMITTED [default READ_COMMITTED]
JDBC fetch size: none
Pool: DatasourceConnectionProviderImpl
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2025-12-09T14:57:21.575+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] o.h.e.t.j.p.i.JtaPlatformInitiator
2025-12-09T14:57:21.601+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] j.LocalContainerEntityManagerFactoryBean
2025-12-09T14:57:21.748+05:30 WARN 159048 --- [Spring_Boot_Rest_API] [ restartedMain] JpaBaseConfiguration$JpaWebConfiguration
configure spring.jpa.open-in-view to disable this warning
2025-12-09T14:57:22.720+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] o.s.boot.tomcat.TomcatWebServer
2025-12-09T14:57:22.731+05:30 INFO 159048 --- [Spring_Boot_Rest_API] [ restartedMain] c.e.demo.SpringBootRestApiApplication

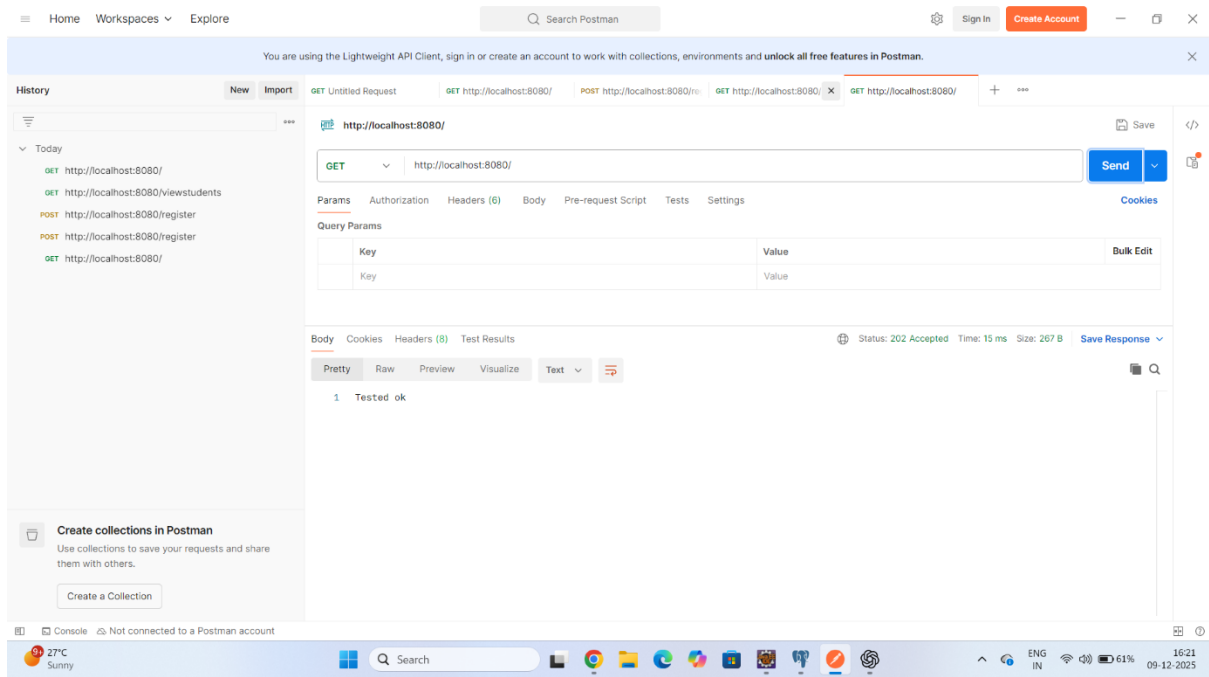
```

Step 7: Testing the API

Now we can test the API using tools like Postman or a web browser.

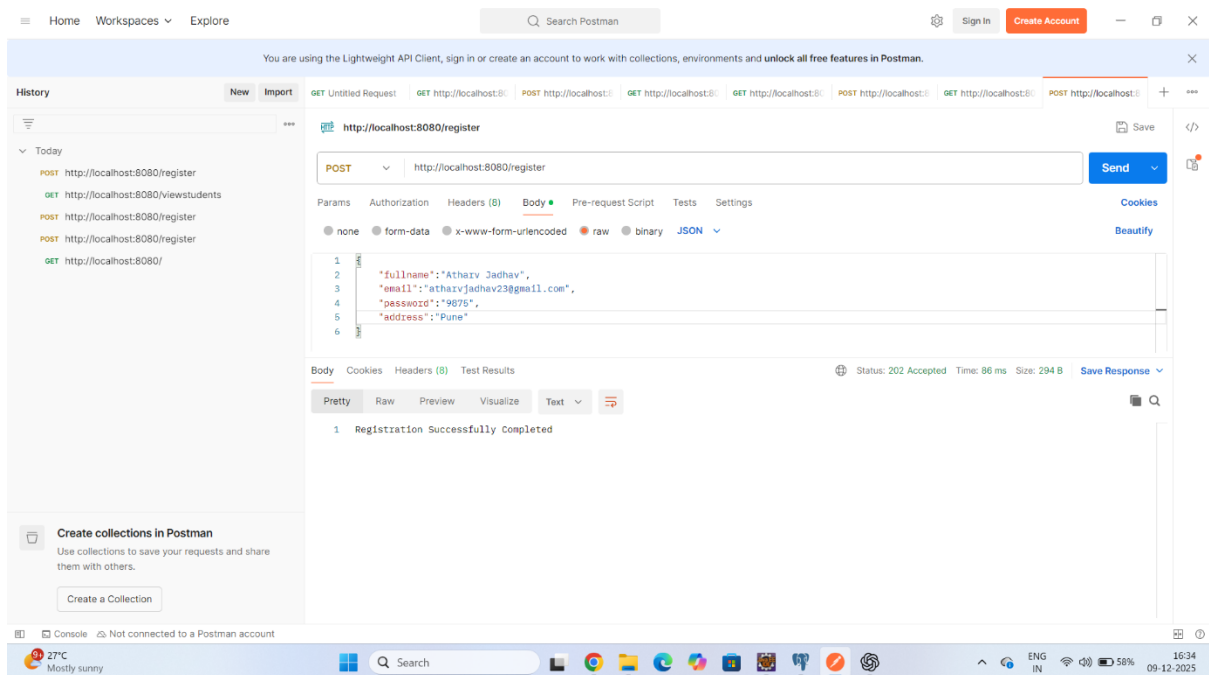
GET Request: Test the API.

URL: `http://localhost:8080/`



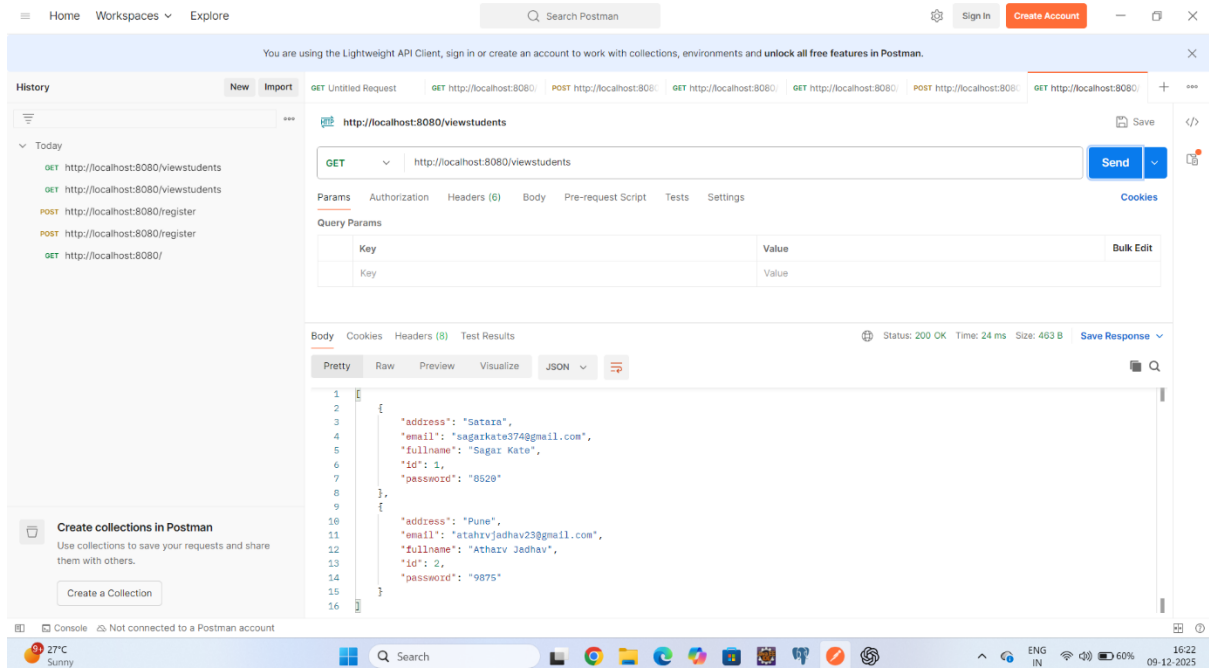
POST Request: Add a new student.

Response:



GET Request: Fetch all students.

URL: <http://localhost:8080/viewstudents>



Web Based -



Database -

	id [PK] bigint	address character varying (255)	email character varying (255)	fullname character varying (255)	password character varying (255)
1	1	Satara	sagarkate374@gmail.com	Sagar Kate	8520
2	6	Pune	atahrvjadhav23@gmail.c...	Atharv Jadhav	9875